A Comparative Multi-Model Framework for Network Intrusion Detection on NSL-KDD: Stacked Ensemble Learning, SHAP Interpretability, and Adversarial Robustness Analysis

Teertha Chatterjee*, Abhyuday Gupta†, Abir Chopra‡, Shaurya Jain§, Vatsal Choudhary¶, Sanjeet Roy‖
*240905082 †240905576 ‡240905059 §240905598 ¶240905248 ‖240905512
Department of Computer Science and Engineering
Email: {240905082, 240905576, 240905059, 240905598, 240905248, 240905512}@institution.edu

***Abstract*—Network Intrusion Detection Systems (NIDS) are a critical line of defence in modern cybersecurity infrastructure. This paper presents a rigorous, reproducible comparative study of seven machine learning classifiers — Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), Support Vector Machine (SVM), K-Nearest Neighbours (KNN), XGBoost, and a novel Stacked Ensemble — evaluated on the NSL-KDD benchmark using the official `KDDTrain+`/`KDDTest+` split for honest, non-inflated reporting. The Stacked Ensemble combines LR, DT, RF, and XGBoost as level-0 base learners with a Logistic Regression meta-learner trained on out-of-fold `predict_proba` outputs via 5-fold Stratified K-Fold cross-validation, yielding 20-dimensional meta-features. On the official test set containing 17 attack types absent from training (16.6% of KDDTest+), XGBoost achieves the highest accuracy of 77.56% and macro-F1 of 52.97%, while the Stacked Ensemble attains 77.40% accuracy. McNemar's test with Bonferroni correction confirms the Stacked Ensemble significantly outperforms RF ($\chi^2 = 450.08,\ p \approx 0$) and that XGBoost marginally but significantly exceeds the Ensemble ($p = 0.019$). SHapley Additive exPlanations (SHAP) with per-class waterfall plots provide novel interpretability insights for all five attack categories. Adversarial robustness under Gaussian feature perturbation reveals KNN as the most robust model ($\Delta{=}-0.053$) while Decision Tree degrades most severely ($\Delta{=}-0.355$). We also demonstrate that SMOTE oversampling significantly improves detection of the severely minority U2R class from F1 $= 0.010$ to F1 $= 0.085$.**

***Index Terms*—Network Intrusion Detection, NSL-KDD, Stacked Ensemble Learning, SHAP, Adversarial Robustness, XGBoost, Random Forest, McNemar's Test, Class Imbalance, SMOTE**

I. Introduction

The proliferation of internet-connected devices and the increasing sophistication of cyber threats make automated network intrusion detection indispensable. A Network Intrusion Detection System (NIDS) monitors network traffic and classifies connections as benign or malicious, ideally distinguishing among distinct attack categories to enable targeted countermeasures.

Machine learning (ML) approaches to NIDS have achieved remarkable results on benchmark datasets, yet a persistent problem in the literature is *evaluation inflation*: the majority of published works report accuracies of 99%+ on NSL-KDD by splitting the training portion (`KDDTrain+`) randomly into 80/20 subsets, thereby testing on attacks the model has already encountered [1]. This methodology is methodologically unsound for any claim about real-world generalisation. The official `KDDTest+` file contains 17 novel attack subtypes absent from `KDDTrain+`, constituting 16.6% of test records; honest evaluation on this partition yields accuracies in the 75–78% range, which is the correct benchmark.

This paper makes the following **novel contributions**:

1) A stacked generalisation ensemble (LR + DT + RF + XGB $\rightarrow$ LR meta) trained with `predict_proba` stacking and 5-fold StratifiedKFold, producing a 20-dimensional probability meta-feature vector.
2) Per-class SHAP waterfall plots for all five NSL-KDD attack categories, the first such class-level interpretability analysis on this benchmark to our knowledge.
3) A systematic multi-model adversarial robustness comparison under Gaussian perturbation of the top-8 SHAP-identified features.
4) McNemar's statistical significance testing with Bonferroni correction across all pairwise comparisons of interest.
5) A rigorous evaluation exclusively on the official `KDDTrain+`/`KDDTest+` partition.

The remainder of this paper is structured as follows. Section II reviews related work. Section III describes the NSL-KDD dataset. Section IV presents our methodology. Section V reports quantitative results and statistical tests. Section VI discusses SHAP interpretability. Section VII covers adversarial robustness. Section VIII addresses minority class improvement. Section XI distils key inferences, and Section XII concludes.

II. Related Work

Intrusion detection with machine learning on KDD Cup 1999 and its refined successor NSL-KDD has been extensively studied. Tavallaee et al. [1] introduced NSL-KDD to address the redundancy and duplication issues of KDD'99. Early works using Decision Trees and Naïve Bayes reported high accuracies

but employed internal random splits [2]. More recent deep-learning approaches including autoencoders [3] and recurrent networks [4] similarly use non-standard splits. Ensemble methods such as Random Forest and Gradient Boosting have been applied to NIDS [5] and consistently outperform single-model baselines. SHAP-based interpretability for network security has gained attention [6], yet per-class waterfall analysis across all five NSL-KDD categories has not previously been reported. Adversarial robustness in NIDS has been explored via FGSM and C&W attacks [7], but Gaussian feature perturbation benchmarking across seven models simultaneously has not been systematically performed. Our work addresses all these gaps on a strictly honest benchmark split.

III. Dataset Description

A. NSL-KDD Overview

NSL-KDD [1] is the de facto benchmark for network intrusion detection research. It is derived from the KDD Cup 1999 dataset with duplicate records removed and difficulty scores attached to each record. The dataset comprises 41 raw features describing individual network connections, plus a label field indicating the traffic type.

B. Train/Test Split

We use exclusively the official partition:

- **KDDTrain+**: 125,973 records $\times$ 42 columns
- **KDDTest+**: 22,544 records $\times$ 42 columns

The `difficulty` column (metadata, not a traffic feature) is dropped, leaving 41 raw features.

C. Attack Category Mapping

All 39 fine-grained attack labels are mapped to five canonical categories. The mapping must reproduce exact canonical counts; any unmapped label is treated as a hard error to prevent silent benchmark corruption:

TABLE I
Class Distribution: KDDTrain+ and KDDTest+

Category	KDDTrain+	KDDTest+
Normal	67,343	9,711
DoS	45,927	7,458
Probe	11,656	2,421
R2L	995	2,754
U2R	52	200
Total	**125,973**	**22,544**

Two mapping decisions are non-obvious: `httptunnel` is assigned to U2R (required to match the canonical U2R count of 200 in KDDTest+), and `snmpguess` is assigned to R2L (SNMP community-string guessing; absent from training). The severe imbalance in the U2R class (52 training samples, ratio 1:1295 vs Normal) is a primary challenge addressed in Section VIII.

D. Feature Engineering and Preprocessing

Three inviolable preprocessing rules are followed:

Rule 1 (Consistent Encoding): One-hot encoding of the three categorical features (`protocol_type`, `service`, `flag`) is performed on the concatenated train+test corpus, then split back. This ensures consistent column cardinality:

$$\mathbf{X}_{\text{enc}}^{(k)} = \text{OHE}\big(\big[\mathbf{X}^{\text{train}};\, \mathbf{X}^{\text{test}}\big]\big)_k\,, \quad k \in \{\text{train}, \text{test}\} \tag{1}$$

Rule 2 (No Data Leakage in Scaling): StandardScaler is fitted exclusively on `KDDTrain+` and applied in transform-only mode to `KDDTest+`:

$$\tilde{x}_j = \frac{x_j - \hat{\mu}_j}{\hat{\sigma}_j}, \quad \hat{\mu}_j = \frac{1}{n_{\text{train}}}\sum_{i=1}^{n_{\text{train}}} x_{ij}, \quad \hat{\sigma}_j = \sqrt{\frac{1}{n_{\text{train}}}\sum_{i=1}^{n_{\text{train}}}(x_{ij} - \hat{\mu}_j)^2} \tag{2}$$

Rule 3 (Official Split Only): `train_test_split` is never applied. The official `KDDTrain+`/`KDDTest+` partition is always used, yielding 122 post-encoding features.

IV. Methodology

A. Model Definitions

Seven models are evaluated, each with `random_state=42` for reproducibility:

1) **Logistic Regression (LR)**: One-vs-rest multi-class with L_2 regularisation ($C = 1.0$, `max_iter`=1000). The decision boundary for class k is:

$$\hat{y} = \arg\max_k\ \sigma\big(\mathbf{w}_k^\top \tilde{\mathbf{x}} + b_k\big)\,, \quad \sigma(z) = \frac{1}{1+e^{-z}} \tag{3}$$

2) **Decision Tree (DT)**: CART algorithm with Gini impurity criterion. Node splits maximise the impurity decrease:

$$\Delta G = G(S) - \frac{|S_L|}{|S|}G(S_L) - \frac{|S_R|}{|S|}G(S_R) \tag{4}$$

where $G(S) = 1 - \sum_{k=1}^{K} p_k^2$ and p_k is the proportion of class k in node S.

3) **Random Forest (RF)**: Bagging ensemble of 100 DTs with feature subsampling $m = \lfloor\sqrt{p}\rfloor$ per split. The ensemble prediction is:

$$\hat{y} = \arg\max_k\ \frac{1}{T}\sum_{t=1}^{T}\mathbf{1}[h_t(\mathbf{x}) = k] \tag{5}$$

4) **SVM (SGD-based)**: Linear SVM trained via stochastic gradient descent (`SGDClassifier`, `loss='hinge'`, L_2 penalty). The primal hinge loss is:

$$\mathcal{L}_{\text{SVM}} = \frac{\lambda}{2}\|\mathbf{w}\|^2 + \frac{1}{n}\sum_{i=1}^{n}\max\big(0,\ 1 - y_i\mathbf{w}^\top\mathbf{x}_i\big) \tag{6}$$

5) **KNN**: k=5 nearest neighbours with Euclidean distance. Classification by plurality vote:

$$\hat{y} = \arg\max_k \sum_{i\in\mathcal{N}_5(\mathbf{x})}\mathbf{1}[y_i = k] \tag{7}$$

6) **XGBoost**: Gradient boosted trees with 100 estimators, learning rate η=0.1, max depth 6. The additive objective at round m is:

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} \ell\Big(y_i,\, \hat{y}_i^{(m-1)} + f_m(\mathbf{x}_i)\Big) + \Omega(f_m) \quad (8)$$

where $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|\mathbf{w}\|^2$ regularises tree complexity (T: leaf count, $\mathbf{w}$: leaf weights).

7) **Stacked Ensemble (Novel Contribution)**: Stacked generalisation [8] with level-0 learners $\{h_{\text{LR}}, h_{\text{DT}}, h_{\text{RF}}, h_{\text{XGB}}\}$ and a Logistic Regression meta-learner. Details in Section IV-B.

B. Stacked Ensemble Architecture

The stacked ensemble produces a 20-dimensional meta-feature vector $\mathbf{z} \in \mathbb{R}^{20}$ (4 models $\times$ 5 class probabilities) via out-of-fold predictions to prevent target leakage:

$$\mathbf{z}_i = \Big[\hat{\mathbf{p}}_{\text{LR}}^{(-i)},\, \hat{\mathbf{p}}_{\text{DT}}^{(-i)},\, \hat{\mathbf{p}}_{\text{RF}}^{(-i)},\, \hat{\mathbf{p}}_{\text{XGB}}^{(-i)}\Big]^\top \quad (9)$$

where $\hat{\mathbf{p}}_m^{(-i)} \in \mathbb{R}^5$ denotes the class probability vector predicted by model m for sample i when i was held out in the k-th fold of a 5-fold Stratified K-Fold cross-validation. The meta-learner g is then trained on $\{(\mathbf{z}_i, y_i)\}_{i=1}^{n_{\text{train}}}$:

$$\hat{y} = g(\mathbf{z}) = \arg\max_k \; \sigma\big(\mathbf{w}_k^\top \mathbf{z} + b_k\big) \quad (10)$$

The StratifiedKFold preserves class proportions within each fold:

$$\frac{|\{i \in \mathcal{F}_k : y_i = c\}|}{|\mathcal{F}_k|} \approx \frac{|\{i : y_i = c\}|}{n}, \quad \forall\, c, k \quad (11)$$

C. Performance Metrics

For a K-class problem with true labels $\mathbf{y}$ and predictions $\hat{\mathbf{y}}$, we report the following metrics:

Weighted F1-score:

$$F_{1,W} = \sum_{k=1}^{K} w_k \cdot F_{1,k}, \quad w_k = \frac{n_k}{n}, \quad F_{1,k} = \frac{2\,P_k\,R_k}{P_k + R_k} \quad (12)$$

Macro F1-score:

$$F_{1,M} = \frac{1}{K}\sum_{k=1}^{K} F_{1,k} \quad (13)$$

Per-class Precision and Recall:

$$P_k = \frac{TP_k}{TP_k + FP_k}, \qquad R_k = \frac{TP_k}{TP_k + FN_k} \quad (14)$$

D. Statistical Significance: McNemar's Test

Let b denote the number of test samples correctly classified by model A but not B, and c vice versa. McNemar's test statistic with continuity correction is:

$$\chi^2 = \frac{(|b-c|-1)^2}{b+c} \sim \chi_1^2 \quad (15)$$

For M simultaneous comparisons, Bonferroni correction adjusts the significance threshold to α/M:

$$p_{\text{corrected}} = \min(M \cdot p_{\text{raw}},\; 1) \quad (16)$$

We apply $M = 2$ (Ensemble vs. RF; Ensemble vs. XGBoost), yielding a corrected significance threshold of 0.025.

E. SHAP Interpretability

For tree ensembles, SHAP provides exact Shapley values [9] via TreeExplainer. The SHAP value for feature j on sample $\mathbf{x}$ is:

$$\phi_j(f, \mathbf{x}) = \sum_{S \subseteq \mathcal{F}\setminus\{j\}} \frac{|S|!\,(|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!}\,[v(S \cup \{j\}) - v(S)] \quad (17)$$

where $\mathcal{F}$ is the full feature set and $v(S)$ is the model prediction conditioned on feature subset S. SHAP satisfies the efficiency axiom:

$$f(\mathbf{x}) = \phi_0 + \sum_{j=1}^{p} \phi_j \quad (18)$$

where $\phi_0 = \mathbb{E}[f(\mathbf{X})]$ is the expected model output.

A stratified SHAP sample of 1,000 instances (200 per class) is drawn from KDDTest+ for interpretability analysis.

F. Adversarial Robustness

We evaluate model robustness by perturbing the top-8 SHAP-identified features with zero-mean Gaussian noise of varying standard deviation σ (in standardised units). The perturbed test instance is:

$$\tilde{\mathbf{x}} = \mathbf{x} + \boldsymbol{\delta}, \quad \delta_j \sim \mathcal{N}(0, \sigma^2) \;\text{ for } j \in \mathcal{S}_8 \quad (19)$$

where $\mathcal{S}_8$ is the set of the 8 most globally important SHAP features. Robustness degradation is quantified as:

$$\Delta_\sigma = \text{Acc}(\sigma = 0) - \text{Acc}(\sigma) \quad (20)$$

evaluated at $\sigma \in \{0, 0.1, 0.2, 0.5, 1.0, 2.0, 5.0\}$ on attack-only samples ($y \neq$ Normal).

G. Minority Class Enhancement: SMOTE

For the severely imbalanced U2R class (52 training instances), we apply Synthetic Minority Over-sampling Technique (SMOTE) [10]. SMOTE generates synthetic samples by interpolating between a minority instance $\mathbf{x}_i$ and one of its k-nearest minority neighbours $\hat{\mathbf{x}}_i$:

$$\mathbf{x}_{\text{syn}} = \mathbf{x}_i + \lambda\,(\hat{\mathbf{x}}_i - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0, 1) \quad (21)$$

with $k = 5$. After SMOTE, all five classes are balanced to 67,343 instances each, yielding a training set of 336,715 samples.

V. Experimental Results

A. Main Performance Comparison

Table II reports accuracy, weighted precision, weighted recall, weighted F1, macro F1, and training time for all 7 models on the official KDDTest+ benchmark.

TABLE II
Comprehensive Results on Official KDDTest+ (22,544 samples)

Model	Acc	P(W)	R(W)	F1(W)	F1(M)	Time
LR	0.7634	0.7494	0.7634	0.7225	0.5389	6.0s
DT	0.7641	0.8161	0.7641	0.7250	0.5289	1.1s
RF	0.7512	0.8052	0.7512	0.7046	0.4790	0.8s
SVM	0.7547	0.7188	0.7547	0.7055	0.4979	5.4s
KNN	0.7461	0.7890	0.7461	0.7037	0.5079	0.1s
XGBoost	**0.7756**	**0.8205**	**0.7756**	**0.7386**	**0.5297**	2.4s
Stack	0.7740	0.8273	0.7740	0.7334	0.5226	40.4s

P(W): weighted precision; R(W): weighted recall; F1(W): weighted F1; F1(M): macro F1.

Fig. 1. Model accuracy comparison on official KDDTest+ with 95% confidence intervals derived from 5-fold cross-validation on KDDTrain+. The Stacked Ensemble bar represents a point estimate (no CV error bar). XGBoost achieves the highest test accuracy of 0.7756.

B. Per-Class Performance

Table III reports precision, recall, and F1 per attack category. All models struggle on R2L and U2R due to the distribution shift between train and test partitions (novel attack subtypes appear only in KDDTest+) and severe class imbalance in training.

TABLE III
Per-Class F1 Scores on KDDTest+ (All 7 Models)

Model	DoS	Normal	Probe	R2L	U2R
LR	0.8859	0.7801	0.7799	0.0660	0.1826
DT	0.8860	0.7882	0.7098	0.1217	0.1389
RF	0.8733	0.7814	0.6935	0.0370	0.0099
SVM	0.8777	0.7762	0.7390	0.0050	0.0917
KNN	0.8666	0.7762	0.6731	0.0740	0.1498
XGBoost	**0.8957**	**0.8012**	0.7101	0.1658	0.0755
Stack	0.8972	0.7956	**0.7372**	0.1155	0.0673
Support	*7458*	*9711*	*2421*	*2754*	*200*

Fig. 2. Confusion matrices for Random Forest (left) and Stacked Ensemble (right) on KDDTest+ (raw counts). The Stacked Ensemble achieves more correct DoS and Normal predictions. Both models mis-classify the majority of R2L and U2R samples due to distribution shift.

C. Cross-Validation Results

Table IV presents 5-fold stratified cross-validation accuracy on the full NSL-KDD dataset (train + test combined). The inflated CV values (97–99%) versus the honest benchmark (75–78%) quantify the magnitude of evaluation inflation introduced by random splitting.

TABLE IV
5-Fold Stratified CV Accuracy (Full NSL-KDD)

Model	μ	σ	95% CI
LR	0.9761	0.00053	[0.9750, 0.9771]
DT	0.9944	0.00046	[0.9935, 0.9953]
RF	0.9956	0.00021	[0.9952, 0.9960]
SVM	0.9671	0.00102	[0.9651, 0.9691]
KNN	0.9901	0.00054	[0.9890, 0.9912]
XGBoost	0.9955	0.00009	[0.9954, 0.9957]
Stack	**0.9958**	0.00027	[0.9953, 0.9964]

Fig. 3. Box plots of 5-fold stratified CV accuracy (full NSL-KDD dataset). All models achieve $> 96\%$ CV accuracy; the spread is very small ($\sigma < 0.001$), confirming low variance. The gap between CV accuracy and KDDTest+ benchmark accuracy (Table II) quantifies the effect of novel attack types in the official test set.

D. Per-Class OOF F1 (Base Ensemble Members)

TABLE V
PER-CLASS OUT-OF-FOLD F1 (5-FOLD, BASE ENSEMBLE MEMBERS)

Model	DoS	Normal	Probe	R2L	U2R
LR	0.9909	0.9786	0.9593	0.7702	0.7594
DT	0.9985	0.9951	0.9930	0.9414	0.7621
RF	0.9991	0.9959	0.9954	0.9472	0.8516
XGBoost	**0.9991**	**0.9959**	0.9938	**0.9504**	**0.8607**

E. Statistical Significance Tests

McNemar's test results are summarised in Table VI. Both comparisons reach statistical significance after Bonferroni correction ($\alpha = 0.025$).

TABLE VI
MCNEMAR'S TEST RESULTS (BONFERRONI-CORRECTED, $M = 2$)

Comparison	b	c	χ^2	p_{corr}
Stack vs. RF	551	36	450.08	$\approx 0^{***}$
Stack vs. XGBoost	78	115	6.715	0.019^{*}

$^{***}p < 0.001$; $^{*}p < 0.05$. Direction: Ensemble $\succ$ RF; XGBoost $\succ$ Ensemble.

F. Learning Curves: Bias-Variance Analysis

Fig. 4. Learning curves for Logistic Regression (left), Random Forest (centre), and XGBoost (right) on KDDTrain+ with 3-fold CV. LR exhibits a slight bias gap; RF and XGBoost converge train and validation accuracy to $> 99\%$ with modest training set sizes, indicating low bias and low variance on the training distribution.

G. ROC and Precision-Recall Curves

Fig. 5. ROC curves (left) and Precision-Recall curves (right) for the Stacked Ensemble on KDDTest+ using One-vs-Rest binarisation. DoS achieves AUC$=0.9746$; Normal AUC$=0.9575$; Probe AUC$=0.9478$. R2L (AUC$=0.8696$) and U2R (AUC$=0.8798$) are harder due to distribution shift and class imbalance. Average Precision (AP) for U2R is 0.1205.

VI. SHAP INTERPRETABILITY ANALYSIS

SHAP TreeExplainer [9] computes exact Shapley values for the Random Forest on 1,000 stratified test samples. SHAP values are in post-StandardScaler units.

A. Global Feature Importance

The global bar plot (mean $|\phi_j|$ averaged over all samples and classes) identifies `src_bytes` as the dominant feature across all attack categories, followed by `dst_host_serror_rate`, `count`, and `dst_host_same_srv_rate`.

Fig. 6. SHAP global beeswarm plot: feature impact distribution across all five attack categories. Each dot represents one SHAP value for one sample; colour encodes feature magnitude (not shown in print). Features are ranked by mean $|\phi_j|$. `src_bytes` dominates globally.

Fig. 7. SHAP mean |value| bar plot decomposed by attack class. `src_bytes` has the highest importance for DoS (blue) and Normal (purple). Probe (pink) depends more heavily on host-diversity features (`dst_host_diff_srv_rate`). R2L and U2R rely on sparse, application-layer features with low global importance.

Fig. 9. SHAP waterfall plot for a representative **Normal** instance. $E[f(\mathbf{X})]$=0.535; $f(\mathbf{x})$=1.0. `src_bytes` (ϕ=+0.073) is the primary driver: low source byte counts combined with a `flag_SF` (successful TCP connection) strongly signal benign traffic. The model aggregates 12 features, each contributing ∼0.02–0.03.

B. Per-Class SHAP Waterfall Analysis (Novel Contribution)

Tables VII summarises the top contributing feature per class from the waterfall plots in Figs. 8–12.

TABLE VII
TOP SHAP FEATURE PER ATTACK CLASS (RANDOM FOREST)

Class	Top Feature	SHAP Value
DoS	`same_srv_rate`	+0.0656
Normal	`src_bytes`	+0.0727
Probe	`dst_host_diff_srv_rate`	+0.1258
R2L	`is_guest_login`	+0.0208
U2R	`service_other`	+0.0070

Fig. 10. SHAP waterfall plot for a **Probe** instance. $E[f(\mathbf{X})]$=0.093; $f(\mathbf{x})$=0.91. `dst_host_diff_srv_rate` (ϕ=+0.126) and `dst_host_same_src_port_rate` (ϕ=+0.110) are the dominant contributors. Probe attacks scan multiple services from a single source port, generating characteristic diversity signatures.

Fig. 8. SHAP waterfall plot for a representative **DoS** test instance. Starting from $E[f(\mathbf{X})]$=0.364, the model reaches $f(\mathbf{x})$=1.0. `same_srv_rate` (ϕ=+0.066) and `dst_host_same_srv_rate` (ϕ=+0.060) are the strongest positive contributors. DoS attacks typically flood a single service, inflating same-service rate metrics.

Fig. 11. SHAP waterfall plot for an **R2L** instance. $E[f(\mathbf{X})]$=0.008; $f(\mathbf{x})$=0.05. The very low base rate reflects severe training imbalance. `is_guest_login` (ϕ=+0.021) and `service_ftp` (ϕ=+0.021) drive the prediction. Several features push against the R2L classification (blue bars), reflecting conflicting evidence common in minority class instances.

Fig. 12. SHAP waterfall plot for a **U2R** instance. $E[f(\mathbf{X})]$=0.000; $f(\mathbf{x})$=0.018. The marginal prediction ($\approx$ 2% probability) reflects near-zero base rate from 52 training samples. `service_other` (ϕ=+0.010) and `dst_host_srv_count` are the top contributors. The waterfall clearly demonstrates that the model lacks sufficient discriminative capacity for this class, motivating the SMOTE experiment in Section VIII.

VII. Adversarial Robustness

The top-8 SHAP-selected features subjected to perturbation are: `src_bytes`, `dst_bytes`, `same_srv_rate`, `dst_host_same_srv_rate`, `srv_serror_rate`, `serror_rate`, `dst_host_serror_rate`, and `count`.

Fig. 13. Adversarial robustness of all 7 models. Detection rate on attack-only test samples (y-axis) as a function of Gaussian noise σ (x-axis). KNN degrades least (Δ=-0.053 at σ=5); Decision Tree degrades most (Δ=-0.355). σ=5 represents a ceiling stress-test (negative byte counts are physically impossible).

TABLE VIII
Adversarial Robustness Summary ($\sigma = 0$ vs $\sigma = 5$)

Model	$\sigma = 0$	$\sigma = 5$	$\boldsymbol{\Delta}$
LR	0.6375	0.3963	-0.2412
DT	0.6135	0.2586	-0.3549
RF	0.5827	0.4053	-0.1774
SVM	0.6193	0.4043	-0.2150
KNN	0.6079	**0.5548**	$\mathbf{-0.0531}$
XGBoost	0.6277	0.3952	-0.2325
Stack	0.6242	0.4136	-0.2106

The KNN model's superior robustness is attributed to its non-parametric, distance-based decision rule: individual feature perturbations shift a sample's local neighbourhood gradually rather than crossing discrete decision boundaries. In contrast, Decision Tree's axis-aligned splits are highly sensitive to perturbations of the exact split features. Random Forest's averaging of 100 trees provides partial robustness ($\Delta = -0.177$), intermediate between single trees and KNN.

VIII. Minority Class Improvement

TABLE IX
U2R and R2L Improvement: RF Variants on KDDTest+

Variant	**Acc**	**MacF1**	**U2R-P**	**U2R-R**	**U2R-F1**
RF (baseline)	0.7512	0.4790	0.333	0.005	0.010
RF (balanced)	0.7468	0.4737	0.333	0.005	0.010
RF (SMOTE)	0.7484	**0.5072**	**0.692**	**0.045**	**0.085**

SMOTE oversampling increases the U2R precision from 0.333 to 0.692 and the U2R F1 from 0.010 to 0.085 — an 8.5$\times$ improvement. While the overall accuracy decreases slightly (0.7512 $\rightarrow$ 0.7484), the macro F1 improves by +2.82 points, confirming better balanced performance. The `class_weight='balanced'` approach provides no benefit over the baseline, suggesting that the challenge for U2R detection is principally the paucity of training examples rather than the loss function weighting. The novel attack subtypes in KDDTest+ (17 attack types absent from KDDTrain+) constitute a hard ceiling for any approach that does not incorporate test-time adaptation.

Post-SMOTE class distribution: all five classes balanced at 67,343 instances, yielding a training set of $5 \times 67,343 = 336,715$ samples.

IX. Hyperparameter Tuning

Grid search (3-fold CV) over the RF parameter space: $\{n_\text{estimators}\} \in \{50, 100\}$, $\{\text{max_depth}\} \in \{10, 20, \text{None}\}$, $\{\text{min_samples_split}\} \in \{2, 5\}$.

The best configuration: `max_depth=None`, `min_samples_split=2`, `n_estimators=50` (CV accuracy = 0.9987, test accuracy = 0.7531, macro F1 = 0.481). The marginal improvement over the baseline RF (test accuracy 0.7512, macro F1 0.479) confirms that the bottleneck is the distribution shift due to novel attack types, not hyperparameter suboptimality.

X. Feature Correlation Analysis

Fig. 14. Lower-triangular Pearson correlation heatmap of the top-20 RF-important features (standardised scale). Strong positive correlations (r>0.7): `flag_SF`–`flag_S0` (r=0.99), `dst_host_serror_rate`–`serror_rate` (r=0.99), `dst_host_same_srv_rate`–`dst_host_srv_count` (r=0.96). These near-collinearities suggest feature redundancy exploitable via dimensionality reduction.

The correlation analysis reveals three tightly coupled feature clusters: (1) error-rate features (`serror_rate`, `srv_serror_rate`, `dst_host_serror_rate`, `dst_host_srv_serror_rate`), (2) service-rate features (`same_srv_rate`, `dst_host_same_srv_rate`, `dst_host_srv_count`), and (3) connection-flag features (`flag_SF`, `flag_S0`). These clusters correspond semantically to SYN-flood DoS patterns, port-scan Probe patterns, and successful/failed connection indicators, respectively.

XI. Inferences and Discussion

A. On Honest Benchmarking

The correct honest result for NSL-KDD is 75–78% accuracy. Publications reporting 99%+ invariably use an 80/20 random split of `KDDTrain+` only, thereby testing on attacks already encountered during training. This practice is methodologically unsound and gives a false impression of real-world robustness. The 17 novel attack subtypes in `KDDTest+` (16.6% of the test set) expose the fundamental *open-world* challenge of NIDS: generalisation to previously unseen attack behaviour.

B. Model Ranking and Trade-offs

XGBoost achieves the best single-model performance (accuracy 0.7756, macro F1 0.5297). The Stacked Ensemble, while significantly better than RF, is marginally but significantly exceeded by XGBoost ($p = 0.019$) — and at 17× higher training cost (40.4s vs. 2.4s). The ensemble's advantage over its base RF learner (McNemar χ^2=450, $p \approx 0$) confirms the theoretical benefit of stacking, but the meta-learner is dominated by XGBoost's strong individual performance.

C. Class-Level Challenges

R2L and U2R consistently achieve low recall across all models. This is explained by two compounding factors: (a) severe class imbalance during training (R2L: 995; U2R: 52) versus test (R2L: 2,754; U2R: 200) and (b) novel attack subtypes in the test set that were never seen during training. The low SHAP values for these classes (Figs. 11, 12) further confirm that the model has not learnt discriminative representations for them.

D. SHAP Insights

The per-class waterfall analysis reveals that different attack categories rely on fundamentally different network primitives:

- DoS attacks are identified primarily through service homogeneity (`same_srv_rate`) and error-rate surge.
- Probes exploit host diversity signatures (`dst_host_diff_srv_rate`).
- Normal traffic is characterised by low `src_bytes` paired with successful TCP establishment (`flag_SF`).
- R2L and U2R depend on application-layer indicators (`is_guest_login`, `service_ftp`, `service_other`) with very small SHAP magnitudes.

These findings motivate class-specific feature engineering as a future direction.

E. Adversarial Robustness Implications

The KNN model's robustness (Δ=−0.053) at the cost of initial accuracy (0.6079 at σ=0 vs. 0.6375 for LR) presents a meaningful trade-off in adversarial deployment contexts. Tree-based methods degrade more severely because their axis-aligned splits are brittle under feature perturbation. A robustness-accuracy Pareto frontier should be considered when selecting models for deployment in potentially adversarial environments.

XII. Conclusion

We have presented a comprehensive, reproducible multi-model framework for network intrusion detection on NSL-KDD, evaluated strictly on the official benchmark split. Our principal findings are:

1) The correct honest accuracy range for NSL-KDD is 75–78%; the 99%+ figures prevalent in the literature arise from methodological inflation via random splits.
2) XGBoost is the best single model (accuracy 0.7756, macro F1 0.5297), and the novel Stacked Ensemble is statistically superior to RF (χ^2=450, $p \approx 0$) but marginally inferior to XGBoost (p=0.019).
3) Per-class SHAP waterfalls provide granular interpretability that global summaries cannot: each attack category relies on a distinct network primitive, enabling class-targeted feature engineering.
4) KNN exhibits the best adversarial robustness under Gaussian perturbation of the top-8 SHAP features; Decision Trees are most vulnerable.
5) SMOTE delivers an 8.5× improvement in U2R F1 (0.010 to 0.085) but does not resolve the hard generalisation problem posed by novel attack subtypes.

Future directions include: deep neural NIDS architectures with attention mechanisms; semantics-preserving adversarial examples for robustness evaluation; continual learning to adapt to emerging attack patterns; and domain adaptation for cross-dataset generalisation.

REFERENCES

[1] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE CISDA*, 2009, pp. 1–6.

[2] S. Revathi and A. Malathi, "A detailed analysis on NSL-KDD dataset using various machine learning techniques for intrusion detection," *Int. J. Eng. Res. Technol.*, vol. 2, no. 12, pp. 1848–1853, 2013.

[3] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, "A deep learning approach for network intrusion detection system," in *Proc. BICT*, 2016, pp. 21–25.

[4] C. Yin, Y. Zhu, J. Fei, and X. He, "A deep learning approach for intrusion detection using recurrent neural networks," *IEEE Access*, vol. 5, pp. 21954–21961, 2017.

[5] H. Zhang, J. Liu, and T. Li, "A network intrusion detection system based on random forest," in *Proc. IEEE ICCSE*, 2019, pp. 918–922.

[6] M. Antwi, D. Gyamfi, and B. Asamoah, "Shapley-based explanations for network intrusion detection," *J. Cybersecurity Priv.*, vol. 1, no. 3, pp. 431–449, 2021.

[7] Z. Lin, Y. Shi, and Z. Xue, "IDSGAN: Generative adversarial networks for attack generation against intrusion detection," in *Proc. PAKDD*, 2022, pp. 79–91.

[8] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241–259, 1992.

[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 4765–4774.

[10] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[11] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM KDD*, 2016, pp. 785–794.

[12] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.